

WORKSHOP PRELIMINARIES

DS

- **Data science (DS):** An interdisciplinary field combining math, statistics, programming, and business knowledge to *turn raw, messy data into useful insights*.
- Traditional DS courses provide **comprehensive coverage** of a programming language (e.g., **Python** or R), building a solid foundation from the ground up
- **Downside:**
 - Overwhelmed by details, students can lose sight of *what matters most* to formulating and implementing a solution to their problem

OUR STRATEGY

- *One day is too short* for developing the knowledge and know-how comprehensively using the traditional approach
- Instead, this workshop simply
 - showcases some **code examples** (primarily in **Python**) with which students can **modify and experiment** to turn them into *their own projects* through learn by doing
 - using *AI as a learning and research partner*

WHY PYTHON?

- Python has become the undisputed industry standard due to an *unmatched ecosystem of pre-built tools*
- In late 1990s,
 - the scientific and academic computing communities adopted Python for mathematics and research
- As the “Big Data” and AI boom later took off,
 - researchers naturally built their new machine learning tools on top of the Python code they already knew

SHELL VS GUI

- Code development in Python/R is often done in a *click-and-select* interface, i.e., **graphical user interface (GUI)**
- A **shell / terminal** lets you
 - interact with the computing environment using a **command line interface (cli)**
- A cli is not as user-friendly as a GUI
 - but can chain code in script files together for *automated/batch processing*

JUPYTER NOTEBOOK

- **Jupyter Notebook/JupyterLab** is an industry-standard GUI for *experimenting with and annotating* Python code,
 - combining code, text, and visualizations in a single document
- It has two types of **cells**:
 - **Code** cells contain code that can be executed to produce output (e.g., tables, plots, numerical or textual results)
 - **Markdown** cells contain text formatted using Markdown syntax

MARKDOWN

- A lightweight markup language for creating formatted text, such as headings, lists, links, and emphasis, using simple *plain-text syntax*
- It is the native language for AI assistants to understand and generate text,
 - making it a natural choice for documenting code and data analysis workflows
- **Quarto**, an industry-standard tool for creating reproducible documents, also uses Markdown to format text
 - e.g., this slidedeck was created with **Quarto using Markdown syntax**

STORY BEHIND JUPYTER NOTEBOOK

- Embedded in the name **Jupyter** are letters representing
 - three major data science programming languages:
 - **Python**, **R**, and **Julia**
- **Julia**, a high-performance language, is a topic for another day
- **R** since its **tidyverse** era has become very user-friendly for data analysis and visualization

R VS PYTHON

- **Python**
 - a general-purpose programming language widely used for data analysis and **machine learning (ML)**
- **R**
 - a programming language specially designed for **statistical computing** and data analysis
- Their capability can be significantly expanded through importing
 - **libraries** (aka. **packages**) specialized in performing certain functions with commands outside the base language

R AND PYTHON

- Today, the data science related functionalities of the two languages have become very similar
 - Can often find similar libraries for both languages
 - e.g., **dplyr** for R and **pandas** for Python are powerful libraries for working with dataframes
 - Can *call R from Python* using the **rpy2** library
 - can also *call Python from R* using the **reticulate** package

FILE FORMATS OF CODE

`myscript` below refers to a script file containing programming code

- **Python:**
 - `.ipynb`: Python code in **Jupyter notebook** format
 - convenient for code development (not for shell use)
 - `.py`: Python code in script format (or marimo notebook as a script)
 - command to execute in shell: `python3 myscript.py`
 - **marimo** notebook offers the *best of both worlds*
- **R:**
 - `.rmd`: R code in **Rmarkdown** notebook format
 - convenient for code development (not for shell use)
 - `.r`: R code in script format
 - command to execute in shell: `Rscript myscript.r`

CLOUD COMPUTING PLATFORMS

The following platforms have free-tier accounts that let you try Python/R conveniently:

- *GitHub Codespaces* has **VS Code** as its GUI
 - native for **GitHub Copilot**
- *Posit cloud* has **RStudio** (for R) as its GUI
 - Can use **GitHub Copilot**
 - but only for inline queries and code generation in the editor

WHY GITHUB, GITHUB CODESPACE, AND GITHUB COPILOT?

- **GitHub** provides spaces to *store, manage, and share* code with others
 - Code files of a project are organized in a *repository (repo)* with *version control* to track changes
- **GitHub Codespace** (built on **VS Code**) provides a *ready-to-use development environment* in the cloud, without requiring the setup of VS Code in a local machine
- **GitHub Copilot** is an *AI assistant* sitting inside Codespace/VS Code to
 - answer your questions (*Ask* mode)
 - formulate plans to implement your ideas (*Plan* mode)
 - act as your agent to execute the plans to generate code and verify the results (*Agent* mode)

GITHUB ECOSYSTEM

GitHub Repository

Storage & Version Control



Your code lives here.

GitHub Codespaces

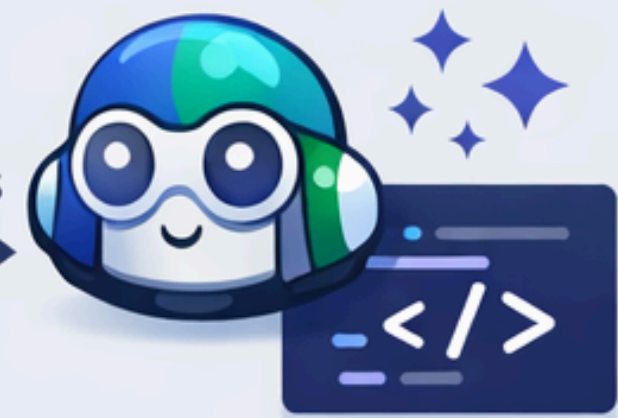
Cloud Coding Environment



Code from anywhere.

GitHub Copilot

AI Code Assistant



AI-powered suggestions.

opens

assists

GITHUB CODESPACE: LAYOUT

- Centre Panel: **Code editor** area
- Tabs on Left Panel:
 - **Explorer**, Search, Source Control, ..., Extensions, ...
- Tabs on Bottom Panel:
 - ... | **Terminal** | ...
| Jupyter (visible when a Jupyter notebook is opened) | ...
- Right Panel: Chat box for **GitHub Copilot**
 - Mode selection (Agent, **Ask**, Plan)
 - Model selection (**Auto** only for free-tier)
- Panel toggle buttons (upper right corner)

WORKSHOP REPO ON GITHUB

- **TrainingWorkshop**: Click this to see the repo's landing page
- A Jupyter notebook in the repo contains code and brief comments to introduce essential Python commands for beginners
- To go through the comments in markdown and execute the code cells,
 - create a **GitHub Codespace** from the repo using the green **Code** button
 - Once ready (i.e., nothing in the Codespace window still spins),
 - find the notebook `prelim2_Jupyter_PythonBasics.ipynb` in the **Explorer** tab on the left panel and click to open the notebook
 - use the **Select Kernel** button (upper right of centre panel) to choose a Python 3.xx kernel
- When hovering over a code cell, you will see an **Execution** button () appearing on the left side of the cell
 - Click it to execute the code cell and see the output below it

THINGS TO NOTE

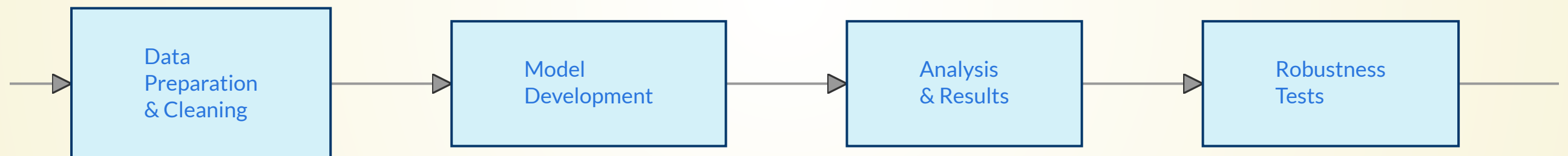
- Be patient as it may take over **12 minutes** to create the Codespace for the first time; subsequent visits require less than 30 seconds to wake it up
 - DON'T start working until the Codespace is ready, or the interruption can result in a **corrupted** Codespace
- *“Do you trust the authors of the files in this folder? Creating a terminal process requires executing code ...”* → click **Trust Folder and Continue**
- *“The workspace has tasks (Run setup on open) defined ... Do you want to allow automatic tasks to run in all trusted workspaces?”* → click **Allow**
- *“Python completions and type diagnostics are unavailable because ...”* → click **Don't Show Again**
- *“OpenCode: enabled VS Code's Agents ...”* → click **Reload Now**
- *“Would you like Visual Studio Code to periodically run “git fetch”?”* → click **Yes**

RSTUDIO PROJECT ON POSIT CLOUD

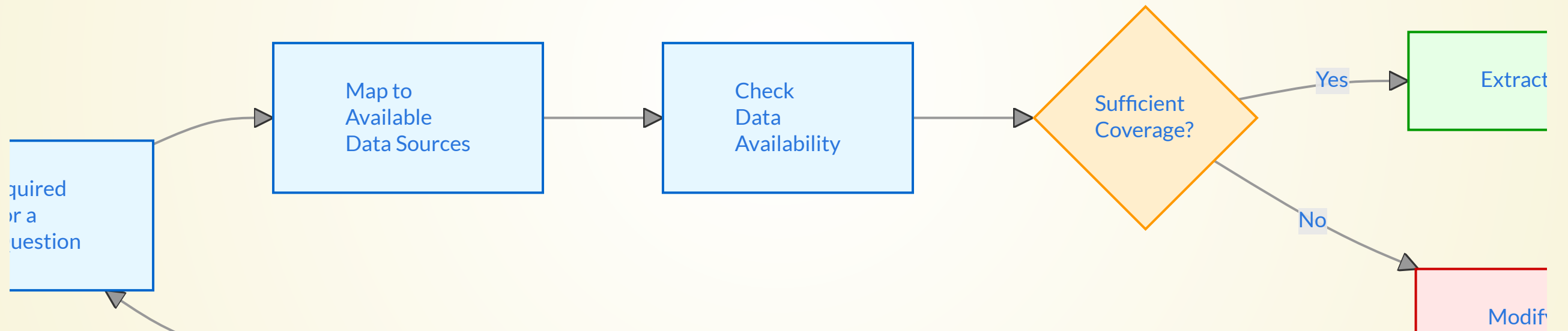
- Project in Workspace on *Posit cloud*
 - is like **Codespace** on GitHub
- See the shared Project:
 - **Result_Tables_Using_R**
- The Rmarkdown notebook `prelim3_ResultTables.Rmd` in the Project contains code to generate the **descriptive statistics, correlation, and regression** tables presented in the second half of this slidedeck
 - The Jupyter notebook `prelim3_ResultTables.ipynb` in the Workshop repo is a Python version of the same code

DATA ANALYSIS WORKFLOW

WORKFLOW OVERVIEW



DATA COLLECTION STRATEGY



DATA SOURCES AVAILABLE AT BAYES

Data Type	Data Source	Content
Financial	Compustat (North America & Global)	Balance sheet, income statement, cash flow data
Financial	Capital IQ: Capital Structure	Detailed capital structure information
Market	CRSP (also, CRSP-Compustat Merged)	Stock prices, returns, trading volume
Market	Datastream	Global equities, economics, commodities data
Banking	Bank Regulatory Database	Bank structure, call reports, holding company financials
...		

EXAMPLE: RESEARCH ON ESG AND FIRM VALUE

- Research Question:
 - *Does superior ESG performance increase firm value for listed companies?*
- Key Variables:
 - Dependent Variable: **Tobin's q** as a proxy for firm value
 - Independent Variable: **ESG score**
 - Control Variables: Firm size, leverage, profitability, etc.
- Sample:
 - Listed companies from **CRSP-Compustat Merged** database
 - Merged with ESG data in the dataset from Ch. 5 of <https://www.greenfinance.education>

TOBIN'S q AND ESG SCORE

- What is Tobin's q ?
- What is **ESG**?

When you do not understand something, *AI is your friend!*
(ask **Claude**)

DATA PREPARATION & CLEANING (BEFORE ANALYSIS)

- Initial assessment:
 - *Summary statistics* for all variables
 - *Missing data* patterns and outlier identification
- Common cleaning procedures:
 - Outlier treatment:
 - e.g., *winsorization*: Typically at 1% and 99%
 - Variable transformation:
 - e.g., *logarithmic* (**log**): Turn a skewed distribution close to normally distributed

MODEL DEVELOPMENT

EXAMPLE: REGRESSION MODELS

- Pooled OLS regression:
 - Basic approach combining *cross-sectional and time-series* data
 - Example:
$$q_{i,t} = \alpha + \beta_1 ESGscore_{i,t} + \beta_2 Controls_{i,t} + \varepsilon_{i,t}$$
- Year and industry indicators:
 - Control for *time trends and industry differences*
 - e.g.,
$$q_{i,t} = \alpha + \beta_1 ESGscore_{i,t} + \beta_2 Controls_{i,t} + \beta_3 \sum Year_t + \beta_4 \sum Industry_i + \varepsilon_{i,t}$$

WHAT ARE YEAR AND INDUSTRY INDICATORS USED IN REGRESSION?

- Ask **Gemini!**

ANALYSIS & RESULTS

EXAMPLE: DESCRIPTIVE STATISTICS TABLE

Variable	N	Mean	Median	Std. Dev.	Min	Max
<i>q</i>	4,200	1.82	1.56	0.94	0.78	4.65
ESGscore	4,200	65.34	67.00	18.45	12.00	95.00
Firm_Size	4,200	8.45	8.32	1.75	4.62	12.85
Leverage	4,200	0.42	0.39	0.21	0.04	0.93

DESCRIPTIVE STATISTICS TABLE

- *How to produce the descriptive statistics table in the attached screenshot (using Python)?*
 - Ask **Qwen** (after uploading the screenshot below)!

ANALYSIS & RESULTS EXAMPLE: DESCRIPTIVE STATISTICS TABLE

Variable	N	Mean	Median	Std. Dev.	Min	Max
<i>q</i>	4,200	1.82	1.56	0.94	0.78	4.65
ESGscore	4,200	65.34	67.00	18.45	12.00	95.00
Firm_Size	4,200	8.45	8.32	1.75	4.62	12.85
Leverage	4,200	0.42	0.39	0.21	0.04	0.93

ANALYSIS & RESULTS

EXAMPLE: CORRELATION TABLE

	<i>q</i>	ESGscore	Firm_Size	Leverage
<i>q</i>	1.000			
ESGscore	0.285***	1.000		
Firm_Size	0.124**	0.356***	1.000	
Leverage	-0.142**	0.034	0.289***	1.000

- *Note:* *** $p < 0.01$, ** $p < 0.05$, * $p < 0.1$

CORRELATION TABLE

- *How to produce the correlation table in the attached screenshot (using Python)?*
 - Ask **Grok Fast** (after uploading the screenshot below)!

ANALYSIS & RESULTS EXAMPLE: CORRELATION TABLE

	<i>q</i>	ESGscore	Firm_Size	Leverage
<i>q</i>	1.000			
ESGscore	0.285***	1.000		
Firm_Size	0.124**	0.356***	1.000	
Leverage	-0.142**	0.034	0.289***	1.000

• Note: *** p<0.01, ** p<0.05, * p<0.1

ANALYSIS & RESULTS

EXAMPLE: REGRESSION TABLE

Variable	Model 1	Model 2	Model 3	Model 4
ESGscore	0.027*** (0.008)	0.024*** (0.007)	0.022*** (0.007)	0.021*** (0.007)
Firm_Size		-0.128*** (0.035)	-0.132*** (0.034)	-0.135*** (0.034)
Leverage		-0.745* (0.371)	-0.732*** (0.154)	-0.728** (0.236)
Industry Indicators	No	No	Yes	Yes
Year Indicators	No	No	No	Yes
Observations	4,200	4,200	4,200	4,200
R-squared	0.081	0.298	0.325	0.337

- Note: Standard errors in parentheses; *** p<0.01, ** p<0.05, * p<0.1

ANALYSIS & RESULTS

EXAMPLE: ROBUSTNESS CHECKS

Variable	Main Results	Alternative ESG Measure	Subsample: Large Firms	Subsample: 2010-2019
ESGscore	0.021*** (0.007)	0.018** (0.008)	0.025*** (0.009)	0.019** (0.008)
Controls	Yes	Yes	Yes	Yes
Industry Indicators	Yes	Yes	Yes	Yes
Year Indicators	Yes	Yes	Yes	Yes
Observations	4,200	4,050	2,340	2,100
R-squared	0.337	0.328	0.356	0.342

- Note: Standard errors in parentheses; *** p<0.01, ** p<0.05, * p<0.1

THANK YOU

- See you at the Workshop!